

Machine Learning Models with Different Inputs

(from ANN, CNN, RNN to Transformer)

Dr. Grantham Pang

1. General Introduction

- Machine learning models are designed around the structure of their input data and the nature of the prediction task.
- The choice of model architecture depends critically on whether inputs are 1-D feature vectors, 2-D grids (images), or time-ordered sequences, and whether the target relationship is linear or nonlinear.

Models: ANN/MLP, CNN, RNN, Transformer

Data: 1D, 2D, 3D, time-sequence

Problems: Regression, Classification, Segmentation ...

1. One-Dimensional (1-D) Array Inputs: Tabular or Feature Vector Data

Problem Setting

- **Input:** A fixed-length 1-D array $x=[x_1, x_2, \dots, x_d]$ ($1 \times d$) representing features (e.g., age, income, sensor readings).
- **Output:**
 - Regression: a continuous value y (e.g., house price).
 - Classification: a discrete label $y \in \{0, 1, \dots, C-1\}$ (e.g., disease present/absent).

Example Applications

- Predicting building energy consumption from sensor features (temperature, occupancy, time of day).
- Classifying sleep stages from a vector of hand-crafted features (mean HRV, spectral power bands, movement count).
- Estimating equipment failure risk from maintenance logs and operating conditions.

SUITABLE MODELS

Linear Models

- **Linear Regression:** Assumes

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_d x_d + \varepsilon.$$

- **Logistic Regression:** For binary classification, where σ is the sigmoid function.

$$P(y = 1|\mathbf{x}) = \sigma(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}),$$

When to use: Relationships are approximately linear; interpretability is important; data is limited.

Nonlinear Models

Multilayer Perceptron (MLP): A feedforward neural network with hidden layers and nonlinear activations (e.g., ReLU). It can approximate any continuous function given sufficient capacity. MLP is providing a **nonlinear functional relationship** between the inputs and output.

- **Tree-based Models:** Random Forests, Gradient Boosting (XGBoost, LightGBM) capture complex interactions and are robust to feature scaling.
- **Support Vector Machines (with kernels):** Map inputs to high-dimensional spaces to find nonlinear decision boundaries.

2. Two-Dimensional (2-D) Array Inputs: Image or Grid Data

Problem Setting

- **Input:** A 2-D array of greyscale or RGB
- **Output:**
 - Image classification: label *yyy* (e.g., "cat", "dog").
 - Semantic segmentation: per-pixel label map.
 - Object detection: bounding boxes + labels.

Example Applications

- Classifying sleep stage spectrograms (time-frequency images from EEG).
- Detecting structural defects from thermal or visual building inspection images.
- Recognizing handwritten digits (MNIST) or medical imaging diagnosis.

SUITABLE MODELS

Convolutional Neural Networks (CNNs)

Key idea: Exploit spatial locality and translation invariance via convolutional filters.

Architecture components:

Convolution + activation (ReLU) layers extract local patterns (edges, textures).

Pooling layers reduce spatial resolution and provide invariance.

Fully connected or global average pooling layers produce final predictions.

Example: LeNet-5, AlexNet, ResNet for image classification.

Worked Example

For a 28×28 grayscale image of a handwritten digit:

- Flatten to 784-D vector $\rightarrow$ MLP: loses spatial structure, many parameters.
- Use CNN: first conv layer with 3×3 filters detects edges; deeper layers detect shapes; final softmax outputs digit class probabilities.

CNNs typically outperform MLPs on image tasks with fewer parameters and better generalization.

3. Sequence Inputs: Time-Stamped or Ordered Data

Problem Setting

- **Input:** A sequence $X=[x_1, x_2, \dots, x_T]$ where each x may be multi-dimensional and is associated with a timestamp or order index.
- **Output:**
 - Next-step prediction: $x(T+1)$ or scalar $y(T+1)$
 - Sequence-to-sequence: map input sequence to output sequence (e.g., translation)
 - Sequence classification: one label for the entire sequence (e.g., sleep stage for a 30-s epoch).

SUITABLE MODELS

Recurrent Neural Networks (RNNs) and Variants

- **RNN:** Maintains hidden state $h_t=f(h_{t-1}, x_t)$ to capture temporal dependencies.
- **LSTM (Long Short-Term Memory):** Adds gates (input, forget, output) to mitigate vanishing gradients and model long-range dependencies.
- **GRU (Gated Recurrent Unit):** Simplified LSTM with reset and update gates; often faster to train.

Transformers for Time Series

- **Key idea:** Use self-attention to weigh all time steps simultaneously, capturing long-range dependencies without recurrence.

Hybrid Models

- **LSTM + Transformer:** LSTM captures local temporal dynamics; Transformer models global dependencies.
- **CNN + LSTM:** 1-D CNN extracts local patterns; LSTM aggregates over time.

Worked Example

Given hourly electricity load for the past 24 hours: $X=[x(t-23),\dots,x(t)]$
predict $x(t+1)$:

- **LSTM:** Processes sequence step-by-step, maintaining memory of daily cycles.
- **Transformer:** Attends to all 24 steps at once; may learn that hour t is most similar to same hour yesterday.
- Empirical studies show Transformers often outperform LSTMs for longer prediction horizons or noisy data.

4. Linear vs Nonlinear Relationships

Linear Relationships

Definition: Output is a linear function of inputs: $y = W^T x + b$

Models: Linear regression, logistic regression

Nonlinear Relationships

Definition: Output depends on inputs in a curved, interactive, or hierarchical way e.g.,

$$y = \sin(x_1) + x_2^2).$$

- Neural networks (MLP, CNN, RNN, Transformer) is essentially providing a nonlinear functional relationship between the inputs and output.
- They can fit complex patterns.
- With abundant parameters, they have the risk of overfitting; requires regularization and sufficient data.

2. Introduction to Transformer

If you use ChatGPT, Google Translate, Gemini, Claude... Perplexity, GitHub Copilot, or even the predictive text on your phone ... you are using a **Transformer**.

It is the architecture introduced in the 2017 paper “**Attention Is All You Need**” by Vaswani et al., and it quietly became the single most important idea in modern AI.

It replaced all older approaches and became the foundation of the entire generative AI era.

3. What is a transformer ?

One Problem Transformers Solve: translate into French
“The animal didn’t cross the street because it was too tired.”

To translate “it” correctly, a model needs to know that “**it**” refers to “**the animal**,” not “the street.” That means the model needs to look *back* across the sentence and connect related words — even when they’re far apart.

Older architectures — **Recurrent Neural Networks (RNNs)** and **LSTMs** — handled sequences one word at a time, left to right, carrying forward a “memory” (hidden state).

The problem: by the time the model reaches “it,” the memory of “animal” has been diluted by everything in between. **Long-range connections** get weaker the farther apart they are. RNN forgot things. Also, RNNs are also inherently slow to train, because word n cannot be processed until word $n-1$ is done — there is **no parallelism**.

The Solution:

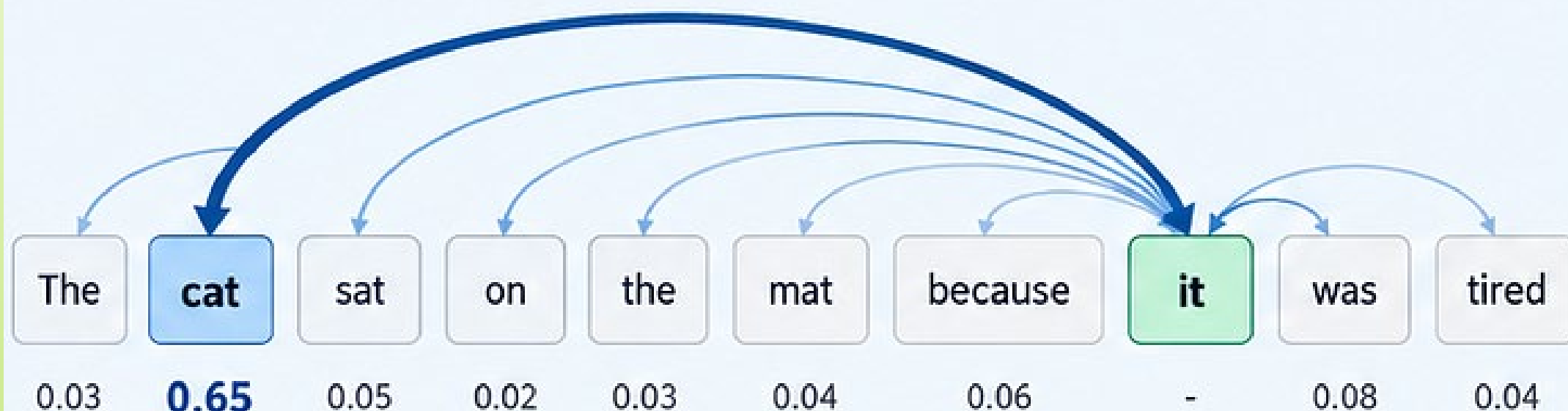
Transformer throws away recurrence entirely, and instead let every word directly “look at” every other word in the sentence in one step. This mechanism is called **self-attention**, and it’s the heart of the entire architecture.

RNN vs. Transformer — Key Differences

Recurrent Neural Network (RNN)	Transformer
Processes tokens one at a time, in order	Processes all tokens at once, in parallel
Struggles with long-range dependencies	Directly connects any two tokens via attention
Training is slow (sequential)	Training is fast (parallelizable on GPUs)
Hidden state is a compressed bottleneck	Every token can attend to every other token
Needs no positional info (order is implicit)	Needs explicit positional encoding

The Attention Mechanism allows LLM to assign different levels of importance to different parts of an input sequence. The ability to selectively focus on information.

When processing the word “it”, the model pays more attention to the relevant word “cat”.



Attention weights for the word “it”

Hence, the **Transformer** is a deep learning architecture introduced in 2017 by Google researchers in the groundbreaking paper “**Attention Is All You Need.**” It completely changed how machines process natural language by:

- **Eliminating sequential processing** (used in RNNs and LSTMs)
- Introducing a powerful concept called self-attention
- Enabling **parallel computation**, allowing models to train faster and understand more complex relationships in text.

Transformers broke the limitation of RNN/LSTM models by:

- Allowing the model to **process all words at once**.
- Instead of recurrence, use **self-attention** to understand the meaning of each word in the context of all others in the sentence.

Pipeline: (from raw text to prediction)

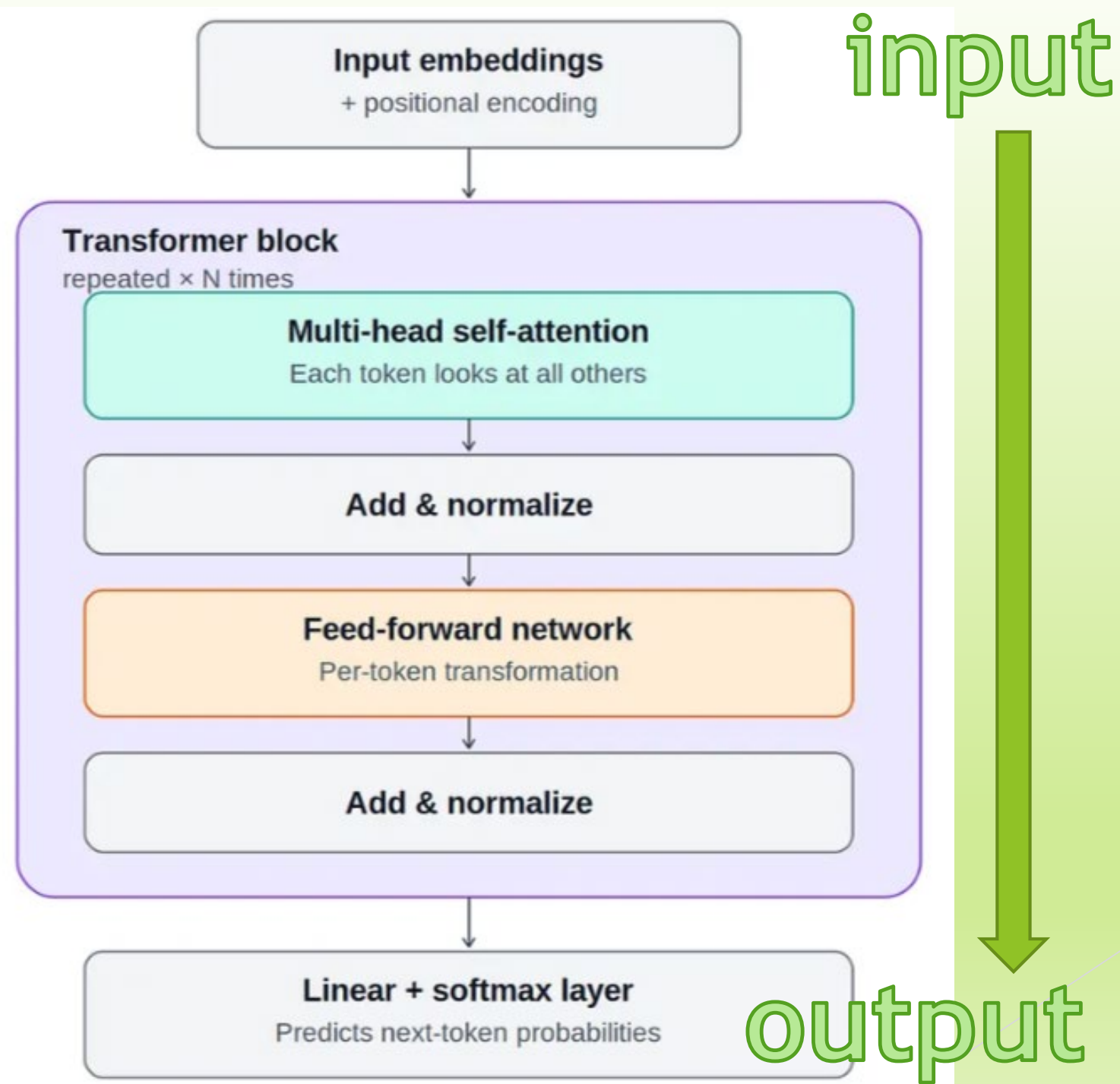
1.Tokenization — split text into smaller units called tokens (often word pieces, not whole words).

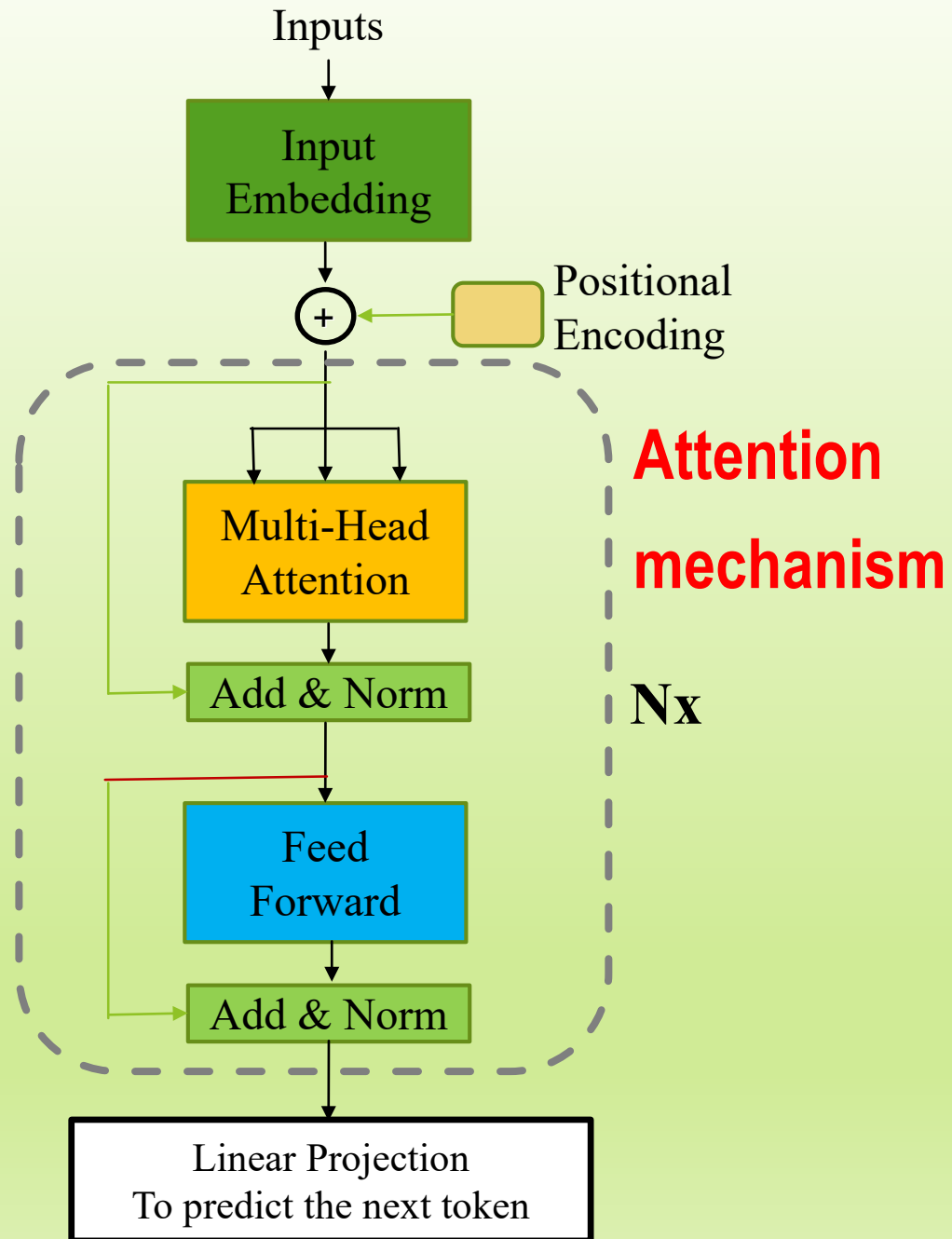
2.Embedding — convert each token into a vector of numbers.

3.Positional encoding — add information about word order.

4.Transformer blocks (stacked N times) — each block runs attention and a feed-forward transformation.

5.Output layer — convert the final representation into a probability for every possible next token.





--- The End ---